Erin Babadilla-Cruz Homework #7 November 1st 2021

1

Q1) Intervals labelled from right to left: In every step, this greedy approach picks the intervals with the latest finishing time among the remaining intervals. Our greedy approach follows the following steps:

1) Sort out the task based on their finishing time (Merge Sort: TC= O(n logn))

2) Pick the task with the latest/maximum (very last) finishing time among all the remaining intervals and discard all the intervals it intersects

3) Repeat step 2 until there are no intervals/tasks left.

Pseudo Code

(1) sort out the intervals based on their finishing time

curr_end = +∞ (INT_max, finishing time of the last picked task, the highest possible finishing time)

(2) for i = 1 to n //traverse all tasks/intervals based on the sorted // order in step one. example curr_end = [Si, Fi] where Si = starting time, Fi = finishing Time

if fi < curr_end
  pick task i
  curr_end = Si //update

return all the picked tasks

TC: O(nlogn) + Θ(n) ≃ O(nlogn)

Proof of Correction

Lemma: For any set of given tasks, there is an optimal subset of them (max-size Pairwise non overlapping) which includes the first task (task w. min finishing time)

Using this Lemma, we will prove the same applies when including the interval w. the latest/max finishing time.

Proof: A: a given set of intervals. I: the task w. max finishing time
S: an optimal subset of A J: the task is S w. max finishing time

Cont on Next page

Gerardo Cruz

Q1

If I belongs to S we are done. Lets suppose S does <u>not</u> include I.

J: the task in S w. max finishing time

In this case, we construct a new optimal subset of tasks S' that includes I. Note that our optimal subset of pairwise non overlapping intervals is NOT necessarily unique.

Observations we can make:

- J does NOT intersect any tasks in S
- Finishing time of I > finishing time of J (by assumption)

$\Rightarrow$ I cannot intersect any other interval from S except J.

Thus, we can construct a new subset S' that excludes J and includes I.

S' is a max size pair wise Non-overlapping subset of $\mathcal{A}$:

- tasks in S' do NOT overlap
- S and S' have the same number of tasks.

Thus, we have constructed an optimal subset of $\mathcal{A}$ that includes I

Δ.

Ezra B Cruz

Q2)

Part A

2) Our greedy approach will consider using the most valuable coins (hierarchy: Quarters, dimes, nickels, pennies) without exceeding the amount n and use as many as those coins as needed, then if there is still a remainder it will consider the next divisible highest coin and continue to do so as long as n (the remaining balance is NOT zero)

Pseudo Code

Let n be the total amount of cents

Num of Q = n/25 // can be zero, this represents Num of Quarters
updated n = n − 25(Num of Q)
Num of D = updated n/10 // see how many dimes
updated n = updated n − 10(Num of D)
Num of N = updated n/5
updated n = updated n − 5(Num of N)
return Num of Q + Num of D + Num of N + updated N (gives total Num of (optimal) coins)
// By the end, updated n ≤ 4, and returns num of pennies

My algorithm considers using the MOST number of greater value coins to optimize the number of coins necessary for our value: First it will calculate what is the most number of quarters we can use, then the most number of dimes, then nickels, and then pennies, to avoid using smaller coins such as pennies if it is NOT necessary. This will return the least number of coins necessary so if n = 91, we know 91/25 = 3 and then 16/10 = 1 so n = 6, n/5 = 1 and n is now 1 meaning we need 3 quarters, 1 dime, 1 nickel and 1 penny, thus

It is NOT possible to make change for 91 cents using less than 6 coins

Eetu Barz

Q2 Part B

2b) Show alg outputs correct result for all pos integers n.

our algorithm accounts for the least number of coins necessary to make change for a value $n > 0$. (pos integers).

If $n == 1$, we can only return 1 coin which is a penny.

Since our algorithm follows the coin value hierarchy (Quarters, dimes, nickels, pennies) we can note the following

- number of quarters: most number of quarters we can use (for any value ≥ 25)

thus number of <u>dimes</u> accounts for a value n which is ≤ 24.

- The number of nickels accounts for any value ≤ 9 since anything higher will be taken care by dimes and or a quarter

- Number of pennies accounts for any value $0 \leq n \leq 4$.

Thus, this algorithm will output the correct result / works for any positive integer n Δ

Coin denomination: face value, how much money its worth ← definition from web.

Q2 Part C

Using the same algorithm (going down based on highest value) Lets say we had coins with value of 1 cent, 5 cents, 8 cents. And lets say we have a value of $n = 12$, then our algorithm would suggest to use 1 8 cent coin and 4 pennies (1 cent coins) BUT the most optimal solution would be two 5 cent coins and two 1 cent coins (4 coins instead of the 5 our algorithm suggested).

Ezra Cruz

Q3: Part A

we know the frog is positioned on the last index of the array (nth index)
In each step, if the frog is positioned on the ith index, then it can make a jump of size at most A[i] towards the beg. of the array: can hop to any of the indices $i, \ldots, i - A[i]$

- we know that if array[i] == 0, and we have NOT reached array[1] aka this first index, we cannot reach the first index since the max num of jumps a frog can make towards the beg of an array is Array[i].
- Also, if Array[i] value is greater than n−i then the frog can directly jump to the very first index since we have more than enough jumps to reach the first index.

Pseudo code

```
JumpIndex = n
for i=n to 1

    // check value
    if (A[i] ≥ (n-i))
        return true // we can directly go to the first index!

    if ((i == 1)
        return True  // we have reached the end

    if (A[i] == 0)
        return False // we have NOT reached the first index (last if statement checks
                     // and cannot traverse any more since A[i] = 0

// By Now we know N is greater than 0 but less than n-i, so
we will jump to that position!

JumpIndex = JumpIndex - A[i]
```

Part B

For loop at most is n iterations.
Each iteration takes O(1) time.
Thus the total tc is O(n) / linear Time.

Ezra Bcruz

Question 4

Greedy Approach: Part A:

1) Sort out the tasks based on their finishing time.

2) Pick the task with minimum finishing time among all the remaining intervals and discard all the intervals it intersects - Prioritize min finishing time

3) Repeat step 2 until NO interval/task is lef.

This greedy alg gives you the maximum size of pair wise Non overlapping intervals.

Pseudo code

1) Sort out intervals based on their finishing time (merge sort)

Curr_end = -∞ // int_Min: finishing time of last picked task

2) for i = 1 to n // traverse all tasks/intervals based on the sorted order in step 1

if s_i > curr_end // does NOT intersect current end

Pick task i

// color it uniquely

curr_end = f_i

{ Checks if the current interval intersects last picked task

return all picked tasks with all unique colors

Q4 Part b

Time Complexity

1) Sorting using Merge Sort: $O(n \log n)$

2) For loop has n iterations (1 to n) so TC = $\Theta(n)$

Total TC = $O(n \log n) + \Theta(n)$ and $O(n \log n)$ is asymptotically larger than $\Theta(n)$ so TC = $O(n \log n)$